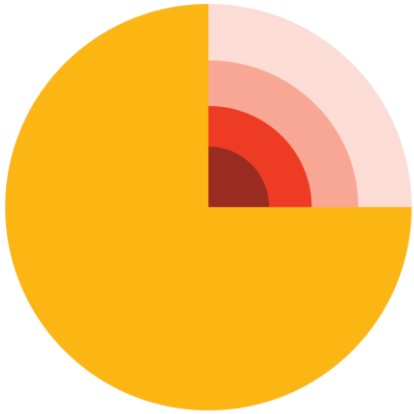


ChEESE

Deliverable
D2.1 First code audit report



Project funded by EuroHPC under the grant agreement No 101093038.

ChEESE-2P Deliverable

D2.1 First code audit report

Document Information

Contract Number	101093038
Project Website	https://cheese2.eu
Contractual Deadline	31/12/2023 (M12)
Activity	WP2
Dissemination Level	PUBLIC
Nature	R

Abstract

Lorem ipsum lorem ipsum lorem ipsum

I. Copyright

Copyright Members of ChEESE collaboration, 2023/2026.

II. Delivery Slip

Role	Name	Partner/Activity
Author	Your Name	Entity
Contributor 1	Name	Entity
Contributor 2	Name	Entity
Reviewer 1	Name	Entity
Reviewer 2	Name	Entity

III.

III. Version control

Version	Date	Description of change	Author/Partner
V1	DD/MM/YYYY	Initial draft	
V2	DD/MM/YYYY		
V3	DD/MM/YYYY	Final version	

IV.

IV. List of Acronyms

Acronym	Description

1 Code ExaHyPE 2

1.1 Brief description of code and use-case

ExaHyPE is an engine for solving hyperbolic PDE systems using high-order discontinuous Galerkin discretization on tree-structured adaptive Cartesian grids. Of particular relevance to ChEESE-2P is ExaSeis, a package to solve seismic wave propagation with kinematic and dynamic rupture earthquake sources on curvilinear meshes. ExaSeis is used in Pilot Demonstrators PD1 (Extreme-scale modeling of seismic hazards) and PD4 (Complex multi-source tsunami modeling). A special focus will be on realizing uncertainty quantification and hazard assessment workflows with ExaSeis.

The ExaHyPE core is developed together by the groups of Tobias Weinzierl (at Durham University, UK) and Michael Bader (TUM). The Durham group leads the work on the underlying Peano framework (for parallel adaptive mesh refinement and overall parallelization approach). TUM is primarily concerned with the development of the ExaSeis solver and with optimization of kernels; in ChEESE-2P a strong focus is put on porting and optimizing ExaHyPE (and ExaSeis) for GPU architectures.

For the performance audit, we consider the LOH1 benchmark, a common benchmark to verify correctness of seismic wave propagation codes. LOH1 models a sediment layer over a base material. The uniform Cartesian grid chosen for this benchmark contains 531,441 cells with a discretization order of 7. Each cell holds 9 unknowns and 3 auxiliary variables. The auxiliary variables remain constant over time. With discretization order 7, this leads to a problem with 3.2 billion degrees of freedom.

1.2 Scaling behavior

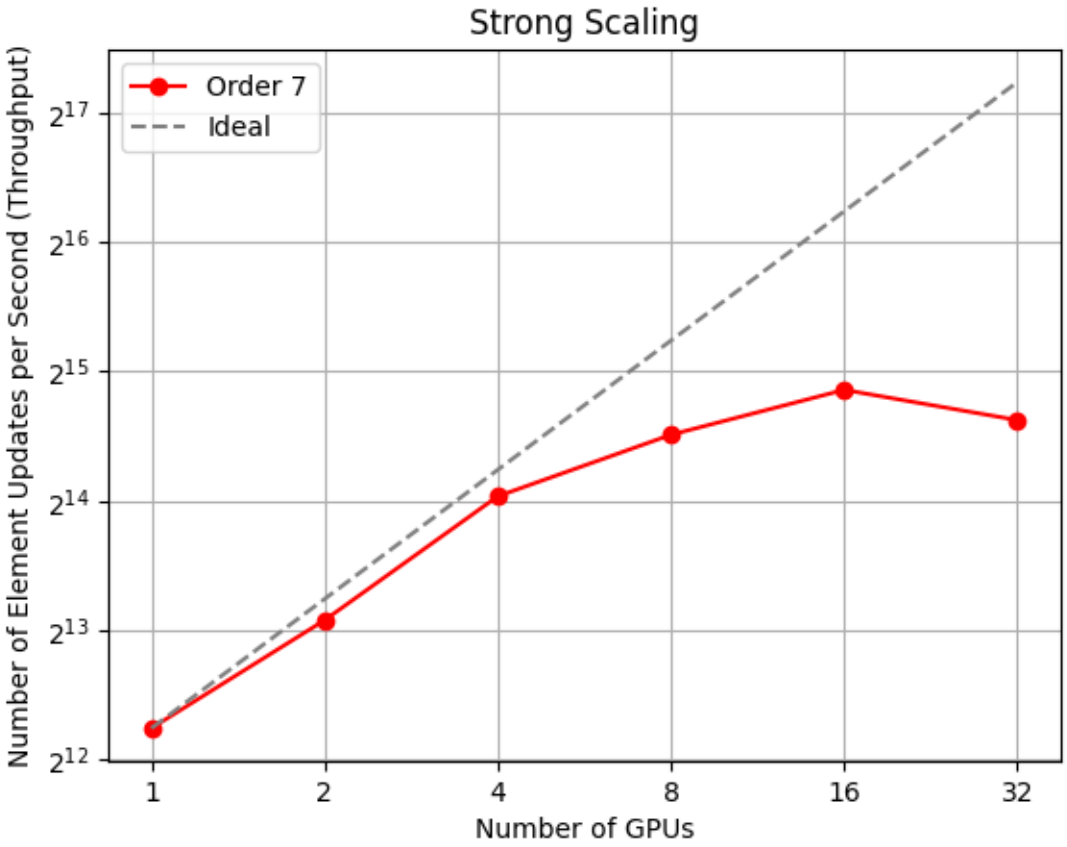
The performance optimization and evaluation performed in ChEESE-1P have revealed a bottleneck in the tree-oriented parallelization approach in the underlying parallel AMR framework Peano. This has led to a major rewrite of Peano and ExaHyPE. The resulting version ExaHyPE 2 is now also more closely integrated into the Peano framework. The ExaSeis package has been ported to the ExaHyPE 2 engine. Its ADER-DG kernels are already optimized for CPUs. For ExaHyPE 2, the focus of ChEESE-2P is now on GPU parallelism, following an OpenMP target offload approach.

The performance assessment in this Deliverable is thus the first evaluation of the ADER-DG seismic wave propagation solver ExaSeis on GPU architectures.

Parallelization approach: Peano/ExaHyPE follow a tree-oriented parallelization approach. MPI partitions are created via space-filling orders on the tree-structured Cartesian grid. Within each MPI rank, the adaptive spacetree can be traversed by multiple threads. A so-called *enclave solver* identifies cells in the adaptive mesh which are of "lower priority" (e.g., cells which are not about to be refined or coarsened or (MPI) boundary cells within the timestep). These *enclave cells* can be offloaded as enclave tasks. To avoid too fine grain tasks, the enclave solver collects these enclave cells until a user-defined threshold is reached and fuses them to a so called "meta cell" which is then offloaded to the GPU. If the mesh is large enough this yields batches of meta cells which are dynamically scheduled and offloaded to all available GPUs.

Known limitations: (1) The threshold for fusing enclave cells is currently user-defined and remains constant over the entire simulation. Finding suitable thresholds requires testing for the specific number of GPUs and also problem size. In a strong scaling scenario, when adding more GPUs, some partitions might not be able to reach the threshold for fusing enclave tasks – GPUs remain underutilized then, which results in a significant drop-off in the strong scaling behavior.
(2) During grid creation, ExaHyPE 2 creates a cascade of spacetrees. It starts with a single space tree assigned to one spacetree-traversal task worked on by one thread. A load balancer takes responsibility to split the spacetree into smaller spacetree during grid creation. Then, multiple spacetree-traversal tasks can start to further create the grid (if applicable, using mesh refinement) in parallel. After grid creation, but before initialization of the problem, the load balancer makes some final adjustment to get the best-balanced load possible. For a static (possibly) adaptive mesh, the load balancer then remains idle during the simulation. Currently, this load-balancing strategy is not yet parallelized, and the grid is completely created in one spacetree. The load balancer only kicks in after the grid is created, which limits the size of the grid that can be created (due to compute time and memory). We excluded this sequential part from the performance analysis.

As an application specific performance metric, the number of element updates per seconds is used. Each rank uses 16 CPU threads per rank available on one node to speed up the fusion of the enclave cells. This number has been fine-tuned and proved to yield the best performance. To setup a strong scaling test a big enough problem size is to be found which can fill at least one GPU's memory. Since 16 CPU threads are necessary per rank/GPU, only one GPU per node is used. Hence, scaling the number of GPUs is achieved by increasing the number of nodes.



1.3 Performance data acquisition

Score-P has been used to obtain traces and performance metrics. The maximal overhead Score-P introduced in the simulation has been calculated to be 2.90%. The used compiler is NVHPC in version 23.5. To also collect OpenMP events with Score-P a patched Score-P installation was needed to support the OMPT interface with NVHPC 23.5.
The focus of analysis (FoA) is the computation of the timestep in the ExaSeis application, i.e., the main loop of the wave propagation kernel.
All acquired data can be reproduced using the code tag 2024ChEESECodeAudit.

1.4 POP performance efficiency metrics

To assess the parallel efficiency of the application, the *multiplicative hybrid model* was chosen. The efficiency metrics were calculated using the advisor plugin of CubeGUI 4.8.1. Hardware counters were measured using Linux Perf.

The following table shows the resulting efficiency metrics for the focus of analysis. Overall, the *Hybrid Parallel Efficiency* significantly declines with scale. The trend highly correlates with the strong scaling plot from section 1.2. From 1 to 2 GPUs there is a 9% loss of parallel efficiency. From 2 to 4 the efficiency remains the same. Beyond 4 GPUs, there is a loss of efficiency of approximately 20% each time the number of GPUs is doubled.

Only a small part of the inefficiency stems from the MPI parallelization as indicated by the almost constant *MPI Parallel Efficiency* around 88%. Most inefficiencies stem from the OpenMP parallelization as *OpenMP Parallel Efficiency* declines down to 23%. Most of the OpenMP inefficiency is caused by a significant load imbalance as indicated by the *OpenMP Load Balance Efficiency* declining down to 29%.

It is important to note, that both the *Hybrid Parallel Efficiency* and the *OpenMP Parallel Efficiency* include all organizational tasks, such as the creation and grouping of enclave tasks, which is executed on CPUs. Many of these organizational tasks are still unoptimized, and, in particular, not parallelized. The respective values are thus not representative of the task distribution to GPUs – the latter cannot be analyzed with Score-P.

POP hybrid multiplicative metrics for ExaSeis-FoA-(timestep)

	1	2	4	8	16	32
Hybrid Parallel Efficiency	0.88	0.79	0.79	0.59	0.4	0.21
Hybrid Load Balance Efficiency	0.91	0.86	0.83	0.71	0.49	0.25
Hybrid Communication Efficiency	0.97	0.92	0.95	0.83	0.8	0.81
MPI Parallel Efficiency	0.94	0.89	0.89	0.89	0.88	0.88
MPI Load Balance	0.94	0.89	0.89	0.89	0.88	0.88
MPI Communication Efficiency	1.0	1.0	1.0	1.0	1.0	1.0
OpenMP Parallel Efficiency	0.93	0.89	0.89	0.66	0.45	0.23
OpenMP Load Balance Efficiency	0.96	0.96	0.93	0.8	0.56	0.29
OpenMP Communication Efficiency	0.97	0.92	0.95	0.83	0.8	0.81
Computational Scaling	1.0	0.95	0.92	0.87	0.82	0.67
Instruction Scaling	1.0	0.96	0.92	0.91	0.85	0.74
IPC_Scaling	1.0	0.99	0.99	0.97	0.97	0.92

1.5 Communication

The metrics report no issue regarding MPI communication as the *MPI Communication Efficiency* is constantly at 100%. All loss of efficiency by communication is attributed to OpenMP as indicated by the *OpenMP Communication Efficiency.* This efficiency captures waiting time caused by dependencies between threads. In total the different threads perform billions of MPI_Iprobe and MPI_Test calls, which on average are shorter than 1 microsecond. This results in huge overhead when tracing the application and hindered us to obtain traces for runs with more than 1 node.

1.6 Computational scalability

The computational scalability of the application is reasonable. At the scale of 32 nodes the *computational scaling* metric reports a value of 67%, which means that the useful computation time is almost 1.5 times longer compared to the reference run on a single node. The longer useful computation time mostly correlates with a larger number of instructions as

the *instruction scaling* metric shows. So there seem to be some redundant computations that increase with scale.

1.7 Observations and recommendations

The most salient observation is the very poor *OpenMP Parallel Efficiency* of 23% at the scale of 32 GPUs, which reports a substantial load imbalance between different OpenMP threads. We suspect that this imbalance is already caused by the administrative tasks to orchestrate the OpenMP offloading to GPUs – further analysis and optimization of these orchestration tasks will be necessary.

Moreover, the *instruction scaling* indicates that redundant operations are performed on the CPU that are not necessary in the reference run with a single GPU on a single node. Here we again suspect influence of non-optimized orchestration tasks and suggest to investigate the cause of this instruction overhead



Project funded by EuroHPC under the grant agreement No 101093038.